7.8 Exercises

For help with exercises, please visit the site, www.wiley.com/college/goodrich.

Reinforcement

R-7.1 Give an algorithm for finding the second-to-last node in a singly linked list in which the last node is indicated by a next reference of None.

R-7.2 Describe a good algorithm for concatenating two singly linked lists L and M, given only references to the first node of each list, into a single list L' that contains all the nodes of L followed by all the nodes of M.

R-7.3 Describe a recursive algorithm that counts the number of nodes in a singly linked list.

R-7.4 Describe in detail how to swap two nodes x and y (and not just their contents) in a singly linked list L given references only to x and y. Repeat this exercise for the case when L is a doubly linked list. Which algorithm takes more time?

R-7.5 Implement a function that counts the number of nodes in a circularly linked list.

R-7.6 Suppose that x and y are references to nodes of circularly linked lists, although not necessarily the same list. Describe a fast algorithm for telling if x and y belong to the same list.

R-7.7 Our CircularQueue class of Section 7.2.2 provides a rotate( ) method that has semantics equivalent to Q.enqueue(Q.dequeue( )), for a nonempty queue. Implement such a method for the LinkedQueue class of Section 7.1.2 without the creation of any new nodes.

R-7.8 Describe a nonrecursive method for finding, by link hopping, the middle node of a doubly linked list with header and trailer sentinels. In the case of an even number of nodes, report the node slightly left of center as the "middle." (Note: This method must only use link hopping; it cannot use a counter.) What is the running time of this method?

R-7.9 Give a fast algorithm for concatenating two doubly linked lists L and M, with header and trailer sentinel nodes, into a single list L'.

R-7.10 There seems to be some redundancy in the repertoire of the positional list ADT, as the operation L.add_first(e) could be enacted by the alternative L.add_before(L.first( ), e). Likewise, L.add_last(e) might be performed as L.add_after(L.last( ), e). Explain why the methods add_first and add_last are necessary.

R-7.11 Implement a function, with calling syntax max(L), that returns the maximum element from a PositionalList instance L containing comparable elements.

R-7.12 Redo the previously problem with max as a method of the PositionalList class, so that calling syntax L.max( ) is supported.

R-7.13 Update the PositionalList class to support an additional method find(e), which returns the position of the (first occurrence of) element e in the list (or None if not found).

R-7.14 Repeat the previous process using recursion. Your method should not contain any loops. How much space does your method use in addition to the space used for L?

R-7.15 Provide support for a __reversed__ method of the PositionalList class that is similar to the given __iter__, but that iterates the elements in *reversed* order.

R-7.16 Describe an implementation of the PositionalList methods add_last and add_before realized by using only methods in the set {is_empty, first, last, prev, next, add_after, and add_first}.

R-7.17 In the FavoritesListMTF class, we rely on public methods of the positional list ADT to move an element of a list at position p to become the first element of the list, while keeping the relative order of the remaining elements unchanged. Internally, that combination of operations causes one node to be removed and a new node to be inserted. Augment the PositionalList class to support a new method, move_to_front(p), that accomplishes this goal more directly, by relinking the existing node.

R-7.18 Given the set of element $\{a,b,c,d,e,f\}$ stored in a list, show the final state of the list, assuming we use the move-to-front heuristic and access the elements according to the following sequence: $(a,b,c,d,e,f,a,c,f,b,d,e)$.

R-7.19 Suppose that we have made kn total accesses to the elements in a list L of n elements, for some integer $k \geq 1$. What are the minimum and maximum number of elements that have been accessed fewer than k times?

R-7.20 Let L be a list of n items maintained according to the move-to-front heuristic. Describe a series of $O(n)$ accesses that will reverse L.

R-7.21 Suppose we have an n-element list L maintained according to the move-to-front heuristic. Describe a sequence of n^2 accesses that is guaranteed to take $\Omega(n^3)$ time to perform on L.

R-7.22 Implement a clear( ) method for the FavoritesList class that returns the list to empty.

R-7.23 Implement a reset_counts( ) method for the FavoritesList class that resets all elements' access counts to zero (while leaving the order of the list unchanged).

Creativity

C-7.24 Give a complete implementation of the stack ADT using a singly linked list that includes a header sentinel.

C-7.25 Give a complete implementation of the queue ADT using a singly linked list that includes a header sentinel.

C-7.26 Implement a method, concatenate(Q2) for the LinkedQueue class that takes all elements of LinkedQueue Q2 and appends them to the end of the original queue. The operation should run in $O(1)$ time and should result in Q2 being an empty queue.

C-7.27 Give a recursive implementation of a singly linked list class, such that an instance of a nonempty list stores its first element and a reference to a list of remaining elements.

C-7.28 Describe a fast recursive algorithm for reversing a singly linked list.

C-7.29 Describe in detail an algorithm for reversing a singly linked list L using only a constant amount of additional space and not using any recursion.

C-7.30 Exercise P-6.35 describes a LeakyStack abstraction. Implement that ADT using a ***singly*** linked list for storage.

C-7.31 Design a *forward list* ADT that abstracts the operations on a singly linked list, much as the positional list ADT abstracts the use of a doubly linked list. Implement a ForwardList class that supports such an ADT.

C-7.32 Design a circular positional list ADT that abstracts a circularly linked list in the same way that the positional list ADT abstracts a doubly linked list, with a notion of a designated "cursor" position within the list.

C-7.33 Modify the _DoublyLinkedBase class to include a reverse method that reverses the order of the list, yet without creating or destroying any nodes.

C-7.34 Modify the PositionalList class to support a method swap(p, q) that causes the underlying nodes referenced by positions p and q to be exchanged for each other. Relink the existing nodes; do not create any new nodes.

C-7.35 To implement the iter method of the PositionalList class, we relied on the convenience of Python's generator syntax and the yield statement. Give an alternative implementation of iter by designing a nested iterator class. (See Section 2.3.4 for discussion of iterators.)

C-7.36 Give a complete implementation of the positional list ADT using a doubly linked list that does not include any sentinel nodes.

C-7.37 Implement a function that accepts a PositionalList L of n integers sorted in nondecreasing order, and another value V, and determines in $O(n)$ time if there are two elements of L that sum precisely to V. The function should return a pair of positions of such elements, if found, or None otherwise.

C-7.38 There is a simple, but inefficient, algorithm, called ***bubble-sort***, for sorting a list L of n comparable elements. This algorithm scans the list $n-1$ times, where, in each scan, the algorithm compares the current element with the next one and swaps them if they are out of order. Implement a bubble_sort function that takes a positional list L as a parameter. What is the running time of this algorithm, assuming the positional list is implemented with a doubly linked list?

C-7.39 To better model a FIFO queue in which entries may be deleted before reaching the front, design a PositionalQueue class that supports the complete queue ADT, yet with enqueue returning a position instance and support for a new method, delete(p), that removes the element associated with position p from the queue. You may use the adapter design pattern (Section 6.1.2), using a PositionalList as your storage.

C-7.40 Describe an efficient method for maintaining a favorites list L, with move-to-front heuristic, such that elements that have not been accessed in the most recent n accesses are automatically purged from the list.

C-7.41 Exercise C-5.29 introduces the notion of a ***natural join*** of two databases. Describe and analyze an efficient algorithm for computing the natural join of a linked list A of n pairs and a linked list B of m pairs.

C-7.42 Write a Scoreboard class that maintains the top 10 scores for a game application using a singly linked list, rather than the array that was used in Section 5.5.1.

C-7.43 Describe a method for performing a ***card shuffle*** of a list of $2n$ elements, by converting it into two lists. A card shuffle is a permutation where a list L is cut into two lists, L_1 and L_2, where L_1 is the first half of L and L_2 is the second half of L, and then these two lists are merged into one by taking the first element in L_1, then the first element in L_2, followed by the second element in L_1, the second element in L_2, and so on.

Projects

P-7.44 Write a simple text editor that stores and displays a string of characters using the positional list ADT, together with a cursor object that highlights a position in this string. A simple interface is to print the string and then to use a second line of output to underline the position of the cursor. Your editor should support the following operations:

- left: Move cursor left one character (do nothing if at beginning).
- right: Move cursor right one character (do nothing if at end).
- insert c: Insert the character c just after the cursor.
- delete: Delete the character just after the cursor (do nothing at end).

P-7.45 An array A is ***sparse*** if most of its entries are empty (i.e., None). A list L can be used to implement such an array efficiently. In particular, for each nonempty cell $A[i]$, we can store an entry (i,e) in L, where e is the element stored at $A[i]$. This approach allows us to represent A using $O(m)$ storage, where m is the number of nonempty entries in A. Provide such a SparseArray class that minimally supports methods __getitem__(j) and __setitem__(j, e) to provide standard indexing operations. Analyze the efficiency of these methods.

P-7.46 Although we have used a doubly linked list to implement the positional list ADT, it is possible to support the ADT with an array-based implementation. The key is to use the composition pattern and store a sequence of position items, where each item stores an element as well as that element's current index in the array. Whenever an element's place in the array is changed, the recorded index in the position must be updated to match. Given a complete class providing such an array-based implementation of the positional list ADT. What is the efficiency of the various operations?

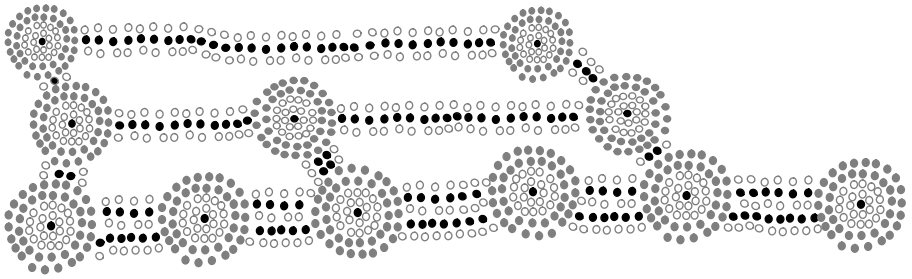
P-7.47 Implement a CardHand class that supports a person arranging a group of cards in his or her hand. The simulator should represent the sequence of cards using a single positional list ADT so that cards of the same suit are kept together. Implement this strategy by means of four "fingers" into the hand, one for each of the suits of hearts, clubs, spades, and diamonds, so that adding a new card to the person's hand or playing a correct card from the hand can be done in constant time. The class should support the following methods:

- add_card(r, s): Add a new card with rank r and suit s to the hand.
- play(s): Remove and return a card of suit s from the player's hand; if there is no card of suit s, then remove and return an arbitrary card from the hand.
- __iter__(): Iterate through all cards currently in the hand.
- all_of_suit(s): Iterate through all cards of suit s that are currently in the hand.

Chapter Notes

A view of data structures as collections (and other principles of object-oriented design) can be found in object-oriented design books by Booch [17], Budd [20], Goldberg and Robson [42], and Liskov and Guttag [71]. Our positional list ADT is derived from the "position" abstraction introduced by Aho, Hopcroft, and Ullman [6], and the list ADT of Wood [104]. Implementations of linked lists are discussed by Knuth [64].

Chapter 10 Maps, Hash Tables, and Skip Lists



Contents

10.1 Maps and Dictionaries 402
10.1.1 The Map ADT 403
10.1.2 Application: Counting Word Frequencies 405
10.1.3 Python's MutableMapping Abstract Base Class 406
10.1.4 Our MapBase Class 407
10.1.5 Simple Unsorted Map Implementation 408
10.2 Hash Tables 410
10.2.1 Hash Functions 411
10.2.2 Collision-Handling Schemes 417
10.2.3 Load Factors, Rehashing, and Efficiency 420
10.2.4 Python Hash Table Implementation 422
10.3 Sorted Maps 427
10.3.1 Sorted Search Tables 428
10.3.2 Two Applications of Sorted Maps 434
10.4 Skip Lists 437
10.4.1 Search and Update Operations in a Skip List 439
10.4.2 Probabilistic Analysis of Skip Lists ★ 443
10.5 Sets, Multisets, and Multimaps 446
10.5.1 The Set ADT 446
10.5.2 Python's MutableSet Abstract Base Class 448
10.5.3 Implementing Sets, Multisets, and Multimaps 450
10.6 Exercises 452